

# 算法设计与分析

## 第4章 贪心算法



- 理解贪心算法的概念
- 掌握贪心算法的基本要素
  - (1) 最优子结构性质
  - (2) 贪心选择性质
- 理解贪心算法与动态规划算法的差异
- 理解贪心算法的一般理论
- 通过应用范例学习贪心设计策略
  - (1) 活动安排问题； (2) 最优装载问题；
  - (3) 哈夫曼编码； (4) 单源最短路径；
  - (5) 最小生成树； (6) 多机调度问题。

顾名思义，**贪心算法**总是作出在当前看来最好的选择。也就是说贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的**局部最优**选择。当然，希望贪心算法得到的最终结果也是整体最优的。虽然贪心算法不能对所有问题都得到整体最优解，但对许多问题它能产生整体最优解。如单源最短路径问题，最小生成树问题等。在一些情况下，即使贪心算法不能得到整体最优解，其最终结果却是最优解的很好近似。

## 4.2 贪心算法的基本要素



### 1. 贪心选择性质

- 所谓**贪心选择性质**是指所求问题的**整体最优解**可以通过一系列**局部最优**的选择，即贪心选择来达到。这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别。
- 动态规划算法通常以**自底向上**的方式解各子问题，而贪心算法则通常以**自顶向下**的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。
- 对于一个具体问题，要确定它是否具有贪心选择性质，必须证明每一步所作的贪心选择最终导致问题的整体最优解。

### 2. 最优子结构性质

当一个问题的最优解包含其子问题的最优解时，称此问题具有**最优子结构性质**。

问题的最优子结构性质是该问题可用动态规划算法或贪心算法求解的关键特征。

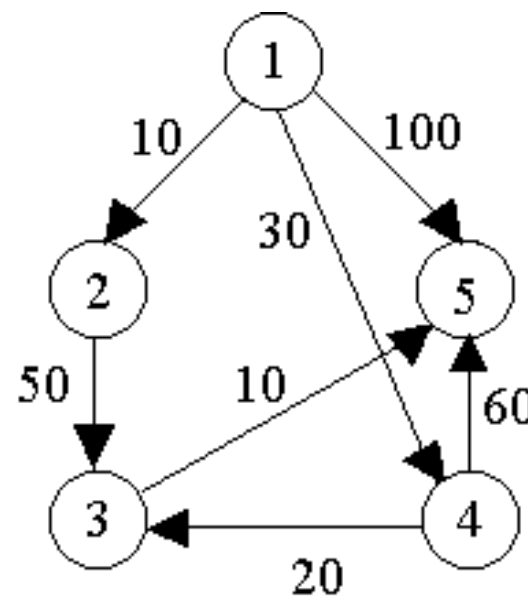
## 4.5 单源最短路径



给定带权有向图 $G=(V,E)$ ，其中每条边的权是非负实数。另外，还给定 $V$ 中的一个顶点，称为**源**。现在要计算从源到所有其它各顶点的**最短路长度**。这里路的长度是指路上各边权之和。这个问题通常称为**单源最短路径问题**。

### 1. 算法基本思想

Dijkstra算法是解单源最短路径问题的贪心算法。



## 4.5 单源最短路径

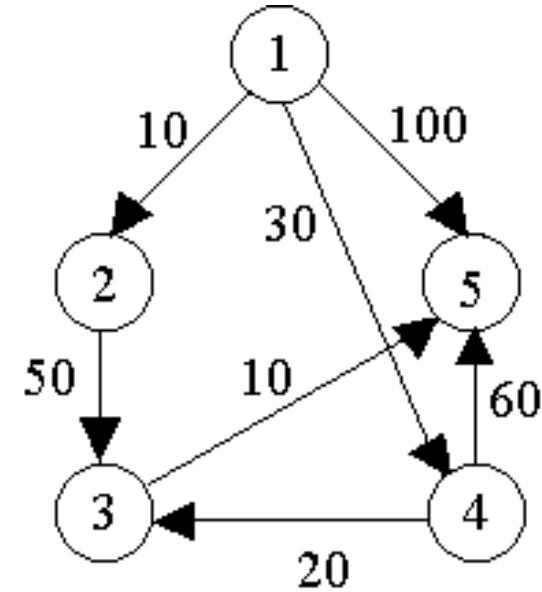


- 其**基本思想**是，设置顶点集合 $S$ 并不断地作**贪心选择**来扩充这个集合。一个顶点属于集合 $S$ 当且仅当从源到该顶点的最短路径长度已知。
- 初始时， $S$ 中仅含有源。设 $u$ 是 $G$ 的某一个顶点，把从源到 $u$ 且**中间只经过 $S$ 中顶点**的路称为从源到 $u$ 的**特殊路径**，并用数组 $dist$ 记录当前每个顶点所对应的最短特殊路径长度。
- Dijkstra算法每次从 $V-S$ 中取出具有最短特殊路长度的顶点 $u$ ，将 $u$ 添加到 $S$ 中，同时对数组 $dist$ 作必要的修改。一旦 $S$ 包含了所有 $V$ 中顶点， $dist$ 就记录了从源到所有其它顶点之间的最短路径长度。

# 4.5 单源最短路径

例如，对下图中的有向图，应用Dijkstra算法计算从源顶点1到其它顶点间最短路径的过程列在下表中。

■ Dijkstra算法的迭代过程：



| 迭代 | S               | u | dist[2] | dist[3] | dist[4] | dist[5] |
|----|-----------------|---|---------|---------|---------|---------|
| 初始 | {1}             | - | 10      | maxint  | 30      | 100     |
| 1  | {1, 2}          | 2 | 10      | 60      | 30      | 100     |
| 2  | {1, 2, 4}       | 4 | 10      | 50      | 30      | 90      |
| 3  | {1, 2, 4, 3}    | 3 | 10      | 50      | 30      | 60      |
| 4  | {1, 2, 4, 3, 5} | 5 | 10      | 50      | 30      | 60      |

# 4.5 单源最短路径



## 2. 算法的正确性和计算复杂性

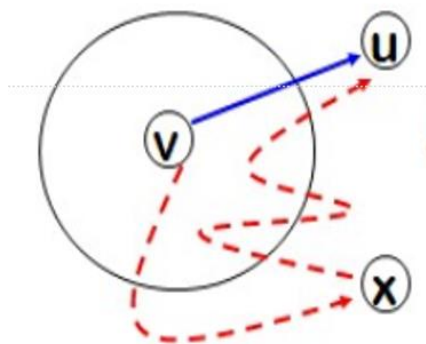
- (1) 贪心选择性质
- (2) 最优子结构性性质
- (3) 计算复杂性

## 4.5 单源最短路径



### (1). 贪心选择性质

- 从 $V-S$ 中选择具有最短特殊路径的顶点 $u$ ，从而确定从源到 $u$ 的最短路径长度 $\text{dist}[u]$ 。
- 为什么从源到 $u$ 没有更短的其他路径？如果存在一条从源到 $u$ 且长度比 $\text{dist}[u]$ 更短的路，设这条路初次走出 $S$ 之外到达的顶点为 $x$ ，然后徘徊于 $S$ 内外若干次，最后离开 $S$ 到达 $u$ 。在这条路上分别记 $d(v, x)$ ,  $d(x, u)$ 和 $d(v, u)$ 为顶点 $v$ 到顶点 $x$ , 顶点 $x$ 到顶点 $u$ , 顶点 $v$ 到顶点 $u$ 的路长。则有：



$$d(v, x) + d(x, u) = d(v, u) \leq \text{dist}[u]$$

$$d(x, u) \geq 0$$

$$\text{dist}[x] \leq d(v, x) \leq \text{dist}[u]$$

**$\text{dist}[x] \leq \text{dist}[u]$ 与 $u$ 是当前贪心选择矛盾！**

### (2). 最优子结构性质

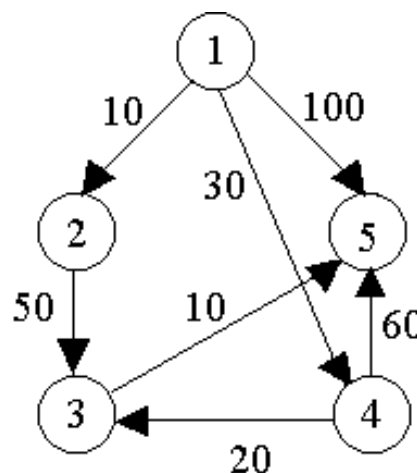
- 如果  $S(i, j) = \{V_i, \dots, V_k, \dots, V_s, \dots, V_j\}$  是从顶点  $i$  到  $j$  的最短路径， $k$  和  $s$  是这条路径上的一个中间顶点，那么  $S(k, s)$  必定是从  $k$  到  $s$  的最短路径。
- 证明：  
假设  $S(i, j) = \{V_i, \dots, V_k, \dots, V_s, \dots, V_j\}$  是从顶点  $i$  到  $j$  的最短路径，则有  $S(i, j) = S(i, k) + S(k, s) + S(s, j)$ 。而  $S(k, s)$  不是从  $k$  到  $s$  的最短距离，那么必定存在另一条从  $k$  到  $s$  的最短路径  $S'(k, s)$ ，那么  $S'(i, j) = S(i, k) + S'(k, s) + S(s, j) < S(i, j)$ 。则与  $S(i, j)$  是从  $i$  到  $j$  的最短路径相矛盾。因此该性质得证。

## 4.5 单源最短路径



### (3). 计算复杂性

对于具有 $n$ 个顶点和 $e$ 条边的带权有向图，如果用带权邻接矩阵表示这个图，那么Dijkstra算法的主循环体需要 $O(n)$ 时间。这个循环需要执行 $n-1$ 次，所以完成循环需要 $O(n^2)$ 时间。算法的其余部分所需要时间不超过 $O(n^2)$ 。



## 4.6 最小生成树



- 设  $G=(V,E)$  是 **无向连通带权** 图，即一个 **网络**。E 中每条边  $(v,w)$  的权为  $c[v][w]$ 。如果  $G$  的子图  $G'$  是一棵包含  $G$  的所有顶点的树，则称  $G'$  为  $G$  的生成树。生成树上各边权的总和称为该生成树的 **耗费**。在  $G$  的所有生成树中，耗费最小的生成树称为  $G$  的 **最小生成树**。
- 网络的最小生成树在实际中有广泛应用。**例如**，在设计通信网络时，用图的顶点表示城市，用边  $(v,w)$  的权  $c[v][w]$  表示建立城市  $v$  和城市  $w$  之间的通信线路所需的费用，则最小生成树就给出了建立通信网络的最经济的方案。

### 1. 最小生成树性质

- 用贪心算法设计策略可以设计出构造最小生成树的有效算法。构造最小生成树的Prim算法和Kruskal算法都可以看作是应用贪心算法设计策略的例子。尽管这2个算法做贪心选择的方式不同，它们都利用了下面的最小生成树性质：
- 设 $G=(V,E)$ 是连通带权图， $U$ 是 $V$ 的真子集。如果 $(u,v) \in E$ ，且 $u \in U$ ， $v \in V-U$ ，且在所有这样的边中， $(u,v)$ 的权 $c[u][v]$ 最小，那么一定存在 $G$ 的一棵最小生成树，它以 $(u,v)$ 为其中一条边。这个性质有时也称为MST性质。

## 4.6 最小生成树



- 设 $G=(V,E)$ 是连通带权图， $U$ 是 $V$ 的真子集。如果 $(u,v) \in E$ ，且 $u \in U$ ， $v \in V-U$ ，且在所有这样的边中， $(u,v)$ 的权 $c[u][v]$ 最小，那么一定存在 $G$ 的一棵最小生成树，它以 $(u,v)$ 为其中一条边。这个性质有时也称为**MST性质**。

原图 $N = \{V, \{E\}\}$

□  $V = \{A, B, C, D, E, F\}$

□  $E = \{(A,B), (A,C), (A,D), \dots\}$

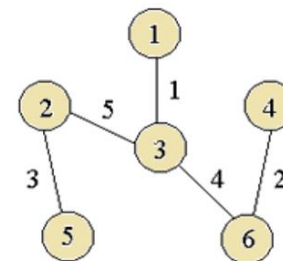
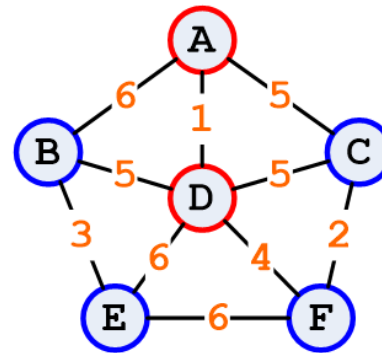
$U \subset V$ ，比如 $U = \{A, D\}$

$V-U = \{B, C, E, F\}$

$\{(A,B), (A,C), (D,B), (D,C), (D,E), (D,F)\}$

权值最小的一条边 =  $(D,F)$

结论是： $N$ 的最小生成树中一定有一棵包含了 $(D,F)$

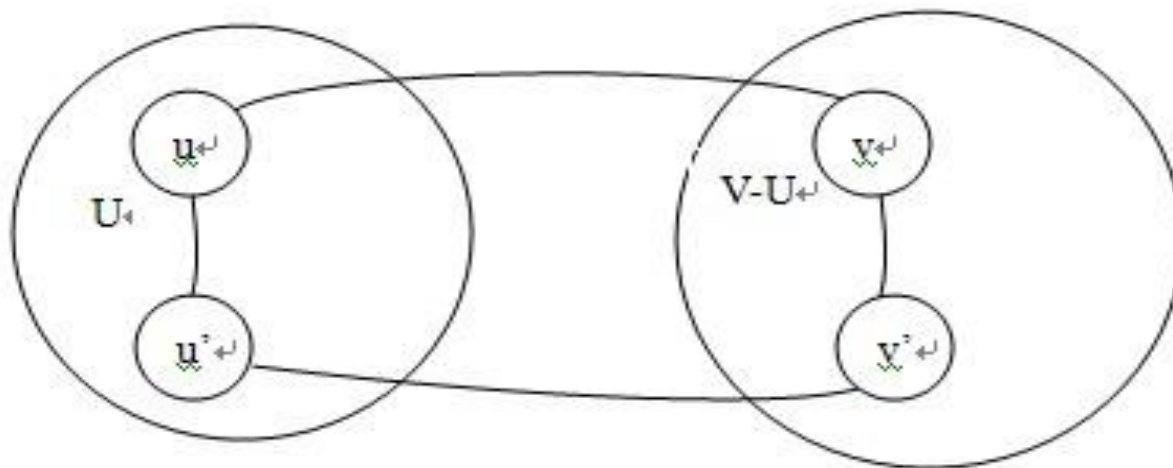


## 4.6 最小生成树



### MST性质证明:

- 假设 $G$ 的任何一颗最小生成树都不包含边 $(u,v)$ 。将边 $(u,v)$ 添加到 $G$ 的一颗最小生成树 $T$ 上, 将产生含有边 $(u,v)$ 的圈, 并且在这个圈上有一条不同于 $(u,v)$ 的边 $(u',v')$ , 使得 $u' \in U$ ,  $v' \in V-U$ 。将边 $(u',v')$ 删去, 得到 $G$ 的另一颗生成树 $T'$ 。由于 $c[u][v] \leq c[u'][v']$ , 所以 $T'$ 的耗费 $\leq T$ 的耗费。于是 $T'$ 是一颗含有边 $(u,v)$ 的最小生成树, 这与假设矛盾。



### 2.Prim算法

- 设 $G=(V,E)$ 是连通带权图， $V=\{1,2,\dots,n\}$ 。
- 构造 $G$ 的最小生成树的Prim算法的**基本思想**是：首先置 $S=\{1\}$ ，然后，只要 $S$ 是 $V$ 的真子集，就作如下的**贪心选择**：选取满足条件 $i\in S$ ， $j\in V-S$ ，且 $c[i][j]$ 最小的边，将顶点 $j$ 添加到 $S$ 中。这个过程一直进行到 $S=V$ 时为止。

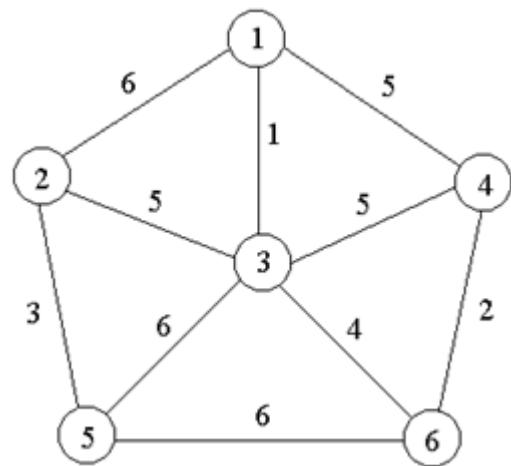
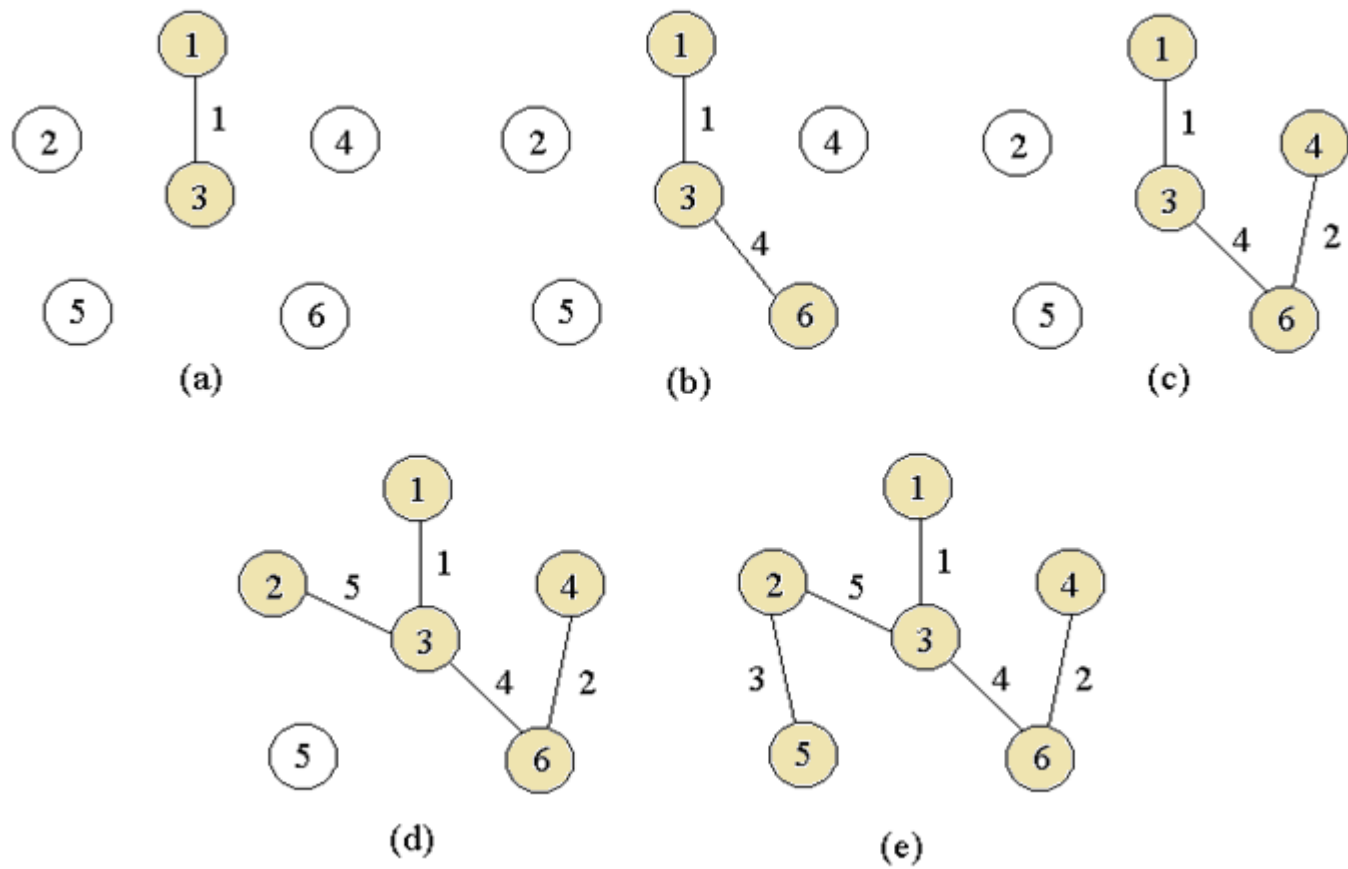
在这个过程中选取到的所有边恰好构成 $G$ 的一棵**最小生成树**。

## 4.6 最小生成树



- 利用最小生成树性质和数学归纳法容易证明，上述算法中的边集合 $T$ 始终包含 $G$ 的某棵最小生成树中的边。因此，在算法结束时， $T$ 中的所有边构成 $G$ 的一棵最小生成树。
- 例如，对于下图中的带权图，按Prim算法选取边的过程如下页图所示。

## 4.6 最小生成树



## 4.6 最小生成树



- 在上述Prim算法中，还应当考虑如何有效地找出满足条件 $i \in S$ ,  $j \in V-S$ , 且权 $c[i][j]$ 最小的边 $(i,j)$ 。实现这个目的的较简单的办法是设置2个数组closest和lowcost。
- $\text{closest}[j]$ 是 $j$ 在 $S$ 中的邻接顶点满足 $c[j][\text{closest}[j]] \leq c[j][k]$ , 即为同 $j$ 最近的节点;  $\text{lowcost}[j] = c[j][\text{closest}[j]]$ 。
- 在Prim算法执行过程中, 先找出 $V-S$ 中使lowcost值最小的顶点 $j$ , 然后根据数组closest选取边 $(j, \text{closest}[j])$ , 最后将 $j$ 添加到 $S$ 中, 并对closest和lowcost作必要的修改。
- 用这个办法实现的Prim算法所需的计算时间为 $O(n^2)$ 。

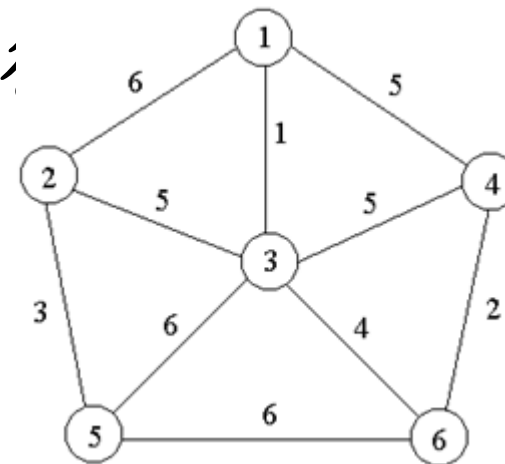
### 3.Kruskal算法

Kruskal算法构造G的最小生成树的基本思想是，首先将G的 $n$ 个顶点看成 $n$ 个孤立的连通分支。将所有的边按权从小到大排序。然后从第一条边开始，依边权递增的顺序查看每一条边，并按下述方法连接2个不同的连通分支：当查看到第 $k$ 条边 $(v,w)$ 时，如果端点 $v$ 和 $w$ 分别是当前2个不同的连通分支 $T_1$ 和 $T_2$ 中的顶点时，就用边 $(v,w)$ 将 $T_1$ 和 $T_2$ 连接成一个连通分支，然后继续查看第 $k+1$ 条边；如果端点 $v$ 和 $w$ 在当前的同一个连通分支中，就直接再查看第 $k+1$ 条边。这个过程一直进行到只剩下一个连通分支时为止。

## 4.6 最小生成树



例如，对右图的连通带权图，按Kruskal算法顺序：  
最小生成树上的边如下图所示。



## 4.6 最小生成树



- 关于集合的一些基本运算可用于实现Kruskal算法。
- 按权的递增顺序查看等价于对优先队列执行removeMin运算。可以用堆实现这个优先队列。
- 对一个由连通分支组成的集合不断进行修改，需要用到抽象数据类型并查集UnionFind所支持的基本运算。
- 当图的边数为 $e$ 时，Kruskal算法所需的计算时间是 $O(e \log e)$ 。

| 算法名   | 普利姆算法           | 克鲁斯卡尔算法        |
|-------|-----------------|----------------|
| 基本思想  | 以 <b>顶点</b> 为对象 | 以 <b>边</b> 为对象 |
| 时间复杂度 | $O(n^2)$        | $O(e \log e)$  |
| 适应范围  | 稠密图             | 稀疏图            |

## 4.7 多机调度问题



- 多机调度问题要求给出一种作业调度方案，使所给的 $n$ 个作业在尽可能短的时间内由 $m$ 台机器加工处理完成。

约定，每个作业均可在任何一台机器上加工处理，但未完工前不允许中断处理。作业不能拆分成更小的子作业。

- 这个问题是NP完全问题，到目前为止还没有有效的解法。对于这一类问题,用贪心选择策略有时可以设计出较好的近似算法。

## 4.7 多机调度问题

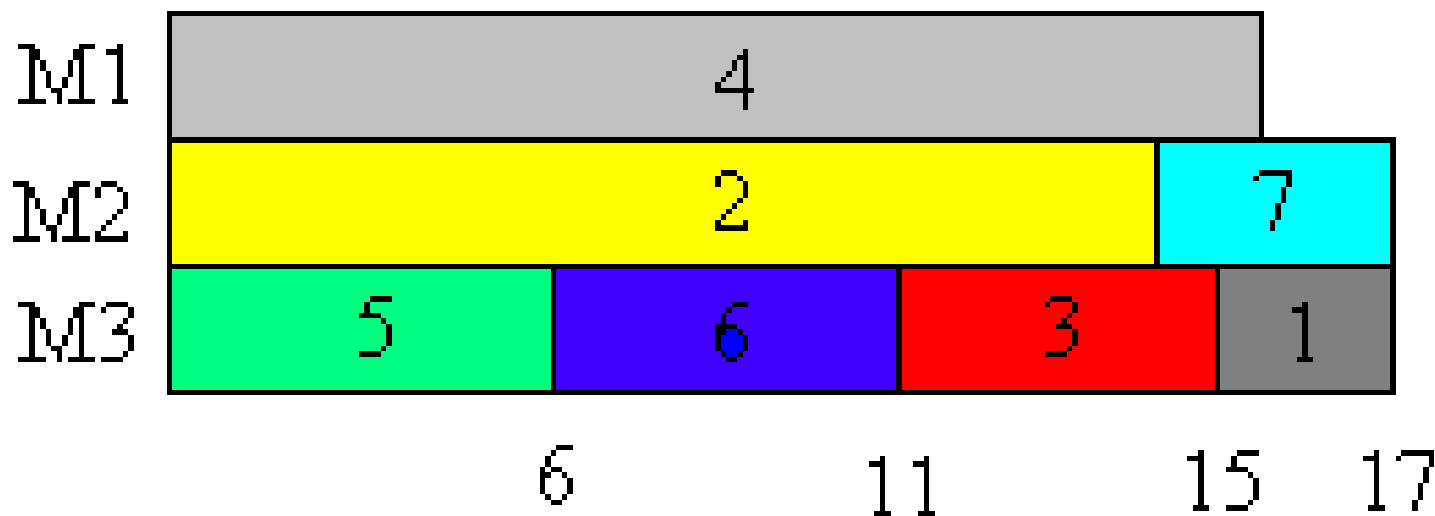


- 采用最长处理时间作业优先的贪心选择策略可以设计出解多机调度问题的较好的近似算法。
- 按此策略，当 $n \leq m$ 时，只要将机器 $i$ 的 $[0, t_i]$ 时间区间分配给作业 $i$ 即可，算法只需要 $O(1)$ 时间。
- 当 $n > m$ 时，首先将 $n$ 个作业依其所需的处理时间从大到小排序。然后依此顺序将作业分配给空闲的处理机。算法所需的计算时间为 $O(n \log n)$ 。

## 4.7 多机调度问题



- 例如，设7个独立作业{1,2,3,4,5,6,7}由3台机器M1, M2和M3加工处理。各作业所需的处理时间分别为{2, 14, 4, 16, 6, 5, 3}。按算法greedy产生的作业调度如下图所示，所需的加工时间为17。



- 1.适用于组合优化问题。求解过程是多步判断，判断的依据是局部最优策略，使目标值达到最大（或最小），与前面的子问题 计算结果无关。
- 2.局部最优策略的选择是算法正确性的关键。
- 3.正确性证明方法：数学归纳法、交换论证。使用数学归纳法主要通过对算法步数或者问题规模进行归纳。
- 如果要证明贪心策略是错误的，只需举出反例。
- 4.自顶向下求解，通过选择将问题归约为小的子问题。
- 5.如果贪心法得不到最优解，可以对问题的输入进行分析，或估计算法的近似比。
- 6.对原始数据排序后，贪心法往往是一轮处理，时间复杂度和空间复杂度都很低。